

Tarea B — Temperatura, top-p y entropía de la distribución de salida

MMIA 6013 · Tarea de práctica para el solver del Taller 03 v2 Entrega: un notebook de Jupyter (`.ipynb`) **ejecutado**, con las salidas visibles.

Un modelo de lenguaje produce, para el siguiente token, un vector de *logits*. Considera un vocabulario de seis tokens, t_0 a t_5 , con estos logits:

Token	t_0	t_1	t_2	t_3	t_4	t_5
Logit	2.0	1.0	0.5	0.2	-1.0	-3.0

Parte 1 — Softmax con temperatura

Implementa en NumPy la función softmax con temperatura $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$, numéricamente estable. Calcula la distribución para $T = 0.5$, $T = 1$ y $T = 2$, y la **entropía en bits** de cada una. Presenta las tres distribuciones y las tres entropías en una tabla, con cuatro decimales.

Parte 2 — Muestreo de núcleo (top-p)

Con la distribución de $T = 1$ de la Parte 1, aplica top-p con $p = 0.9$: ordena los tokens por probabilidad, conserva el conjunto mínimo cuya probabilidad acumulada alcanza 0.9 y renormaliza. Reporta qué tokens sobreviven y la distribución renormalizada.

Parte 3 — Comprobación empírica

Toma 10 000 muestras de la distribución renormalizada de la Parte 2 con `numpy.random.default_rng(0)`. Compara las frecuencias observadas con las probabilidades teóricas en un gráfico de barras y calcula la divergencia KL (en bits) de la distribución empírica respecto de la teórica.

Parte 4 — Pregunta conceptual

Explica qué le pasa a la distribución cuando $T \rightarrow 0$ y cuando $T \rightarrow \infty$, y por qué $T \rightarrow 0$ equivale a la decodificación voraz (*greedy*). Usa las entropías de la Parte 1 como evidencia.

Formato

Un solo notebook, ejecutado de principio a fin sin errores, con una celda de Markdown que encabece cada parte. Las cifras se calculan en el notebook, no se escriben a mano.